

AADK 11: Task 2 — Responsive List & Image Loading Strategy

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 11

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Responsive Behavior Plan

Layout Adaptation

The list will adapt depending on the **screen size and orientation**.

Device Type	Layout
Phone (Portrait)	Single-column scrolling list
Phone (Landscape)	Two-column grid
Tablet	Two or three-column grid

This approach ensures that the UI uses the available screen space efficiently and improves readability.

Phone Layout (Portrait)



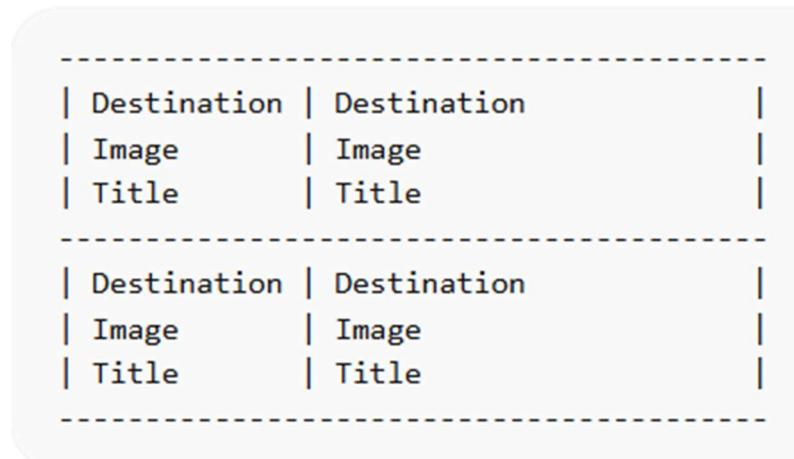
Compose Component Used:

LazyColumn

Reason:

Phones in portrait mode have limited width, so a **vertical list is easier to read and scroll**.

Tablet / Landscape Layout



Destination	Destination	
Image	Image	
Title	Title	
Destination	Destination	
Image	Image	
Title	Title	

Compose Component Used:

LazyVerticalGrid

Reason:

Tablets and landscape screens have more horizontal space, so a **grid layout allows multiple items to be displayed simultaneously**, improving usability.

Compose Layout Strategy

Example concept:

```
if (screenWidth < 600.dp) {  
    LazyColumn {  
        items(destinations) { item ->  
            DestinationCard(item)  
        }  
    }  
}
```

```
    }  
  } else {  
    LazyVerticalGrid(columns = GridCells.Fixed(2)) {  
      items(destinations) { item ->  
        DestinationCard(item)  
      }  
    }  
  }  
}
```

2. Image Loading Strategy

Image Source

Images will be loaded from **local drawable resources**.

Example:

R.drawable.destination1

R.drawable.destination2

R.drawable.destination3

Local images are faster and more reliable for this project.

Image Loading Method

Images will be displayed using:

painterResource(id = destination.imageResId)

If internet images were used, the **Coil library** would be used with:

```
AsyncImage(  
    model = imageUrl,  
    contentDescription = null  
)
```

Placeholder Strategy

While an image is loading:

- Show a **default placeholder image**
- Example: `R.drawable.placeholder`

If image loading fails:

- Show an **error image**
- Example: `R.drawable.image_error`

Example concept:

```
AsyncImage(  
    model = imageUrl,  
    placeholder = painterResource(R.drawable.placeholder),  
    error = painterResource(R.drawable.image_error),  
    contentDescription = "Destination Image"  
)
```

Performance Tips

1. **Use appropriately sized images**
Avoid very large images to reduce memory usage.
 2. **Use lazy lists (LazyColumn / LazyVerticalGrid)**
Only visible items are loaded, improving performance.
 3. **Avoid loading full-resolution images unnecessarily**
Resize images to the required UI dimensions.
 4. **Cache images when loading from internet**
Libraries like Coil automatically cache images.
-

3. Resource & Memory Considerations

Image Size Target

Images will be optimized to approximately:

200 × 200 dp

This size provides good visual quality without using excessive memory.

Handling Large Lists

To manage large datasets efficiently:

- Use **LazyColumn** / **LazyVerticalGrid** so only visible items are rendered.
- Implement **pagination or incremental loading** for very large lists.
- Avoid loading all images at once.

This helps reduce **memory usage and improves scrolling performance**.

4. Reflection

Image-rich scrollable lists can create challenges such as slow loading times, high memory usage, and laggy scrolling if images are too large. Responsive layouts add complexity because the UI must adapt to different screen sizes and orientations while maintaining a consistent design. Efficient image loading and lazy list components are important to ensure smooth performance. Optimizing images and managing resources properly helps maintain a fast and responsive mobile user experience.
